

New Paltz
STATE UNIVERSITY OF NEW YORK

Traffic Light Project

Tanvir Khan, Javier Padilla

State University of New York at New Paltz

EGC331-01 - Microcontroller System Design

Professor Michael Otis

5 May 2020

Abstract: Create a working traffic light that has four modes that are able to use Rural with pedestrian crossing, Rural without pedestrian crossing, Urban with pedestrian crossing, and Urban without pedestrian crossing. This project uses various components such as an LCD, seven segment display, potentiometer, led's, and various resistors, and etc. The IDE that was used was keil version 4.7 on a Ti Tiva TM4C123GH6PM board.

Introduction	3
Theory	3
Design	5
Multifunctional Process	5
Rural Traffic Light	6
Rural with Pedestrian crossing light	8
Urban Only Traffic light	10
Urban with pedestrian crossing light	11
Materials and ports initiation locations:	13
Code:	14
Problems and solutions:	15
Results and Conclusion	15
References	16
Appendices	16
Appendix A: Whole Project Code	16

Introduction

The purpose of this project was to create a traffic light that can use different modes, use a pedestrian crossing system, and an LCD to display the mode. At first the Rural traffic light was designed. Then the pedestrian traffic light was designed. After that we were trying to display some words on the LCD display. Then we had to figure out how to use a multifunction switch that can control the whole traffic light. When designing the multifunctional user control, the urban traffic light was thought to change modes from a Rural traffic light. Finally, we had to put everything together. Basically a traffic light is used to control oncoming traffic and is used everywhere there are cars driving. As explained in the driver's handbook, there are three lights in one direction of the traffic and they are red, yellow, and green. Green means to go, yellow means to slow down, and lastly Red means to stop. [1]

Theory

This project was broken down into four main parts: Traffic light, pedestrian traffic light, display LCD commands, and multi function process. Each part can be related to a specific purpose in software generally. All of the parts were combined together to work in a coherent process.

Basically a rural traffic light is used mostly in the country area, where there is not a lot of traffic to control. The rural traffic light uses sensors to detect oncoming traffic. In a general traffic light there are four directions These consist of the directions North, South, East, and West. Since the North and South direction have the same traffic they would be connected to both lights. Meaning the North and South direction will have the same output. The same goes for the East

and West direction. The West and East direction will also have the same output since they have the same oncoming traffic.

The pedestrian control is based on the user pushing the button and waiting for the traffic light to finish its current cycle and then moving on, so that the pedestrian can cross. A pedestrian control is needed to make the roads as safe as possible for the people that walk on foot and do not drive. This way less accidents can occur and allows freedom for pedestrians to walk. When the pedestrian is enabled to walk, then all the traffic lights for all directions should be indicated as red. When pedestrians are walking there is a time limit that is displayed at the opposite side of where they're walking to indicate how much time they have until the traffic light resumes its process and does not allow any more pedestrians to walk. As explained in Pedestrian Crossing Time, the crosswalk should be able to display a countdown for the pedestrians as well as flash when the timer is about to end. [2] This design will be explained later in the rural traffic design.

As it says in its name, an urban traffic light is used in urban areas. The urban traffic light is used in the city area, where there is a lot of traffic to control. The urban traffic light does not use sensors unlike the rural traffic light. The Urban traffic light is automated, and goes through a sequence of steps over and over again. Meaning that the Urban traffic light is an automated process that goes through steps in a loop.

The LCD is a liquid crystal display that is used for displaying characters or words. The LCD is powered and grounded. Additionally the LCD needs a potentiometer to control the brightness of the lighting on the LCD. The LCD is a very powerful tool to relay messages to others. The LCD is a good device for communicating to others as well. The LCD that's used in this project only has 32 characters that it can display. There are two rows and each row can display eight characters.

The seven segment display is used to display numbers and letters, which is also a powerful tool used to relay messages. The Seven segment display led's can either be enabled through ground or power. In this case the LED's are enabled through ground. The reason it's called a seven segment display is because it has seven bar shaped LED's. The Seven segment display can only display numbers and letters. The numbers that it can display are from zero to nine. The letters that it can display are A, B, C, D, E, and F. The letters correspond to Hexadecimal values.

Design

Multifunctional Process

There were four modes that were made with the multifunctional part. There were four switches controlling each mode. There was one set of four slide switches that were in one compartment. The first switch was for Rural and crosswalk mode. The second switch was for Rural only mode. The third switch was for Urban and Crosswalk mode. Lastly, the fourth switch was for Urban only mode. The next few sections are ordered like this to talk about each process before combining both the pedestrian and the preferred traffic light. Using what we learned from

two previous labs, we were able to incorporate it into this project. The Urban and Rural Design lab showed us how to create urban and rural traffic lights and how they behave. [3] For the second lab, Seven Segment Display and LCD, It gave an understanding on how to display values on an LCD as well as create a timer on a Seven Segment display. [4]

Rural Traffic Light

The Rural traffic light was based on a state table. Before talking about the Rural with pedestrian mode, the rural only mode is going to be explained. The Rural traffic light in our project has four states. This state table can be seen below in *Figure 1*.

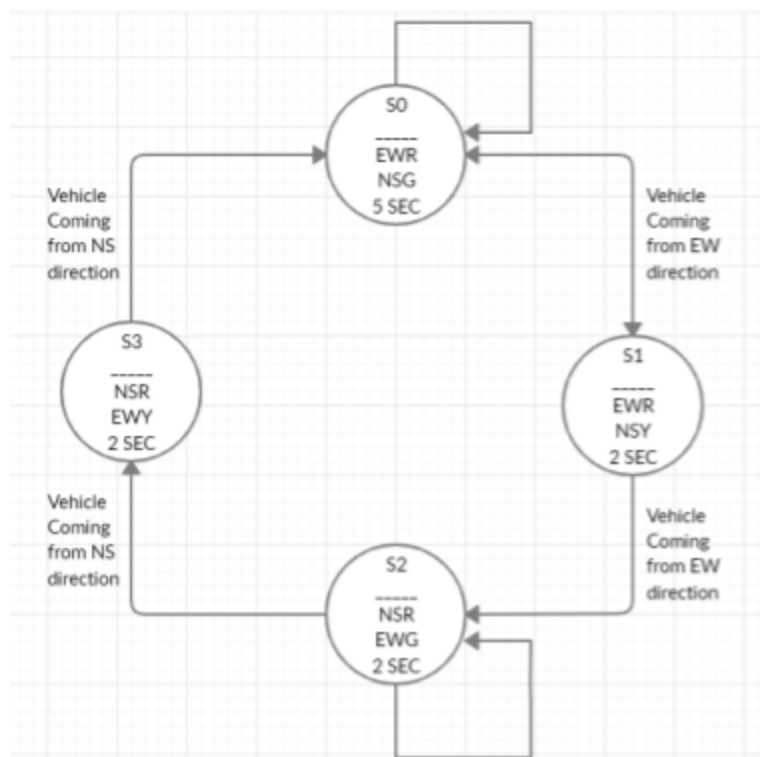


Figure 1. Shows the States of the Rural only traffic lights in a state diagram.

The state diagram above shows four states. Being that the rural traffic light has to start somewhere, I went with north and south being green and east and west being red, which is in state zero. Then it waits for five seconds. Then it senses if there is a vehicle coming from the east and west direction, if it does it goes to north and south being yellow, and east and west being red, which is state one. If there's no cars coming from the east and west direction then it will loop back to itself and keep doing state zero. Then after going through state one, it keeps going and goes through state two, with north and south being red, and east and west being green. It loops to itself if it does not detect any cars coming from the north and south direction. If it doesn't detect a car coming from the north and south direction it will go through state three with, north and south being red and, east and west being yellow. Then it goes back to state 0, which is the initial state back to north and south being green and, east and west being red. It loops through this sequence continuously.

Rural with Pedestrian crossing light

The only difference between the state diagram in *Figure 1* for the Rural and pedestrian crossing light would be that the pedestrian crossing is added. When adding the pedestrian crossing light another condition is added between state zero and one from *Figure 1*. The condition is that if it detects a pedestrian, and if it does it will go through state one, then go to a new state which would be called state four. State four would be called the red, red condition. Meaning that in both directions the traffic light would be red. The same thing would happen for state two, but a little different from what happened from state zero to state two. From state two it would detect if there was a pedestrian coming and if it did it would go to state three and then a

new state is created called state five which is the same as state four but it goes back to state zero.

The Rural with crosswalk state diagram can be seen in *Figure 2*.

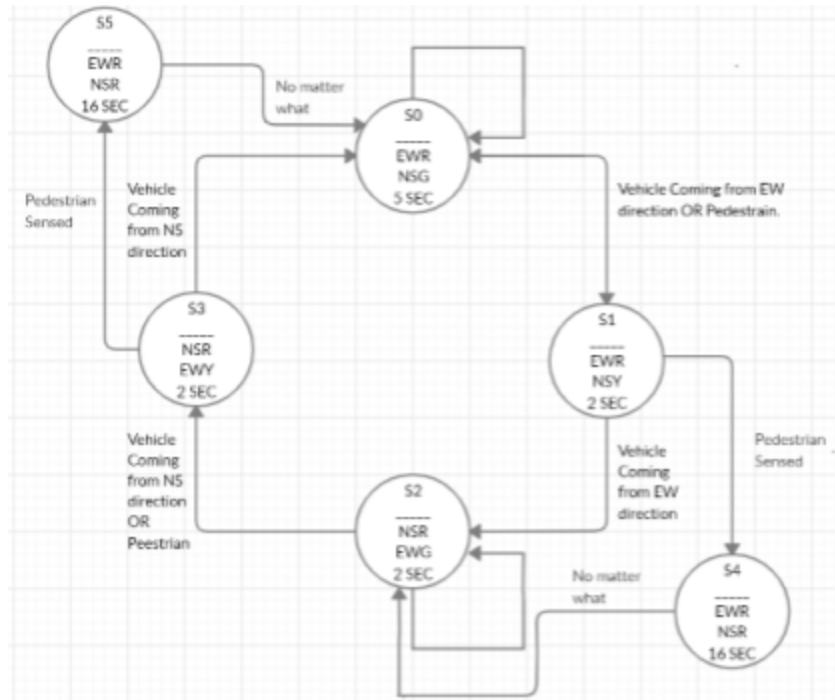


Figure 2. Shows the States of the Rural with crosswalk traffic light in a state diagram.

The state diagram in *Figure 2* starts out at state zero. It will then sense if there is a pedestrian or car coming from the NS direction, and if it does it will go to the next state. We have to note that the pedestrian crosswalk takes higher precedence than the cars coming from either direction. So if a pedestrian is sensed it will automatically go from state zero, then to state two, then to state four. If a car was sensed and no pedestrian was sensed in the north and south direction then it goes from state zero to state one, then to state two. From there, it goes to state two to state three, then state five, then back to state zero, when it detects a pedestrian in the east

and west direction. If there is a car being sensed from the east and west direction and no pedestrians who pushed the button, then it will go from state two to state three, to state zero, back to the initial state. It keeps doing this in a continuous loop.

When state four and state five are enabled, then every direction turns red. Additionally the seven segment display counts down from 9 to 0, then it flashes '0' three times. If statements were used to light up the corresponding port number with the leds on the segments of the seven segment display.

Urban Only Traffic light

In the Urban traffic, it is similar to the Rural only traffic, but the only difference is it goes through each state automatically. Meaning it does not wait to sense any vehicles from all of the directions. It is a continuous running loop program which does not care about any switches being turned on. *Figure 3* shows a state diagram of the urban only traffic light.

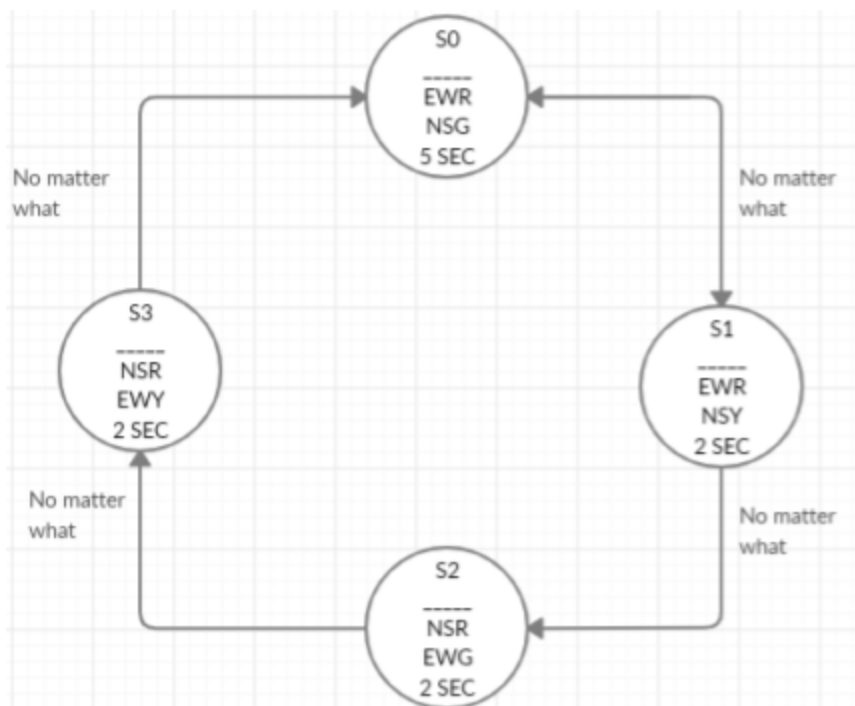


Figure 3. Shows the States of the Urban only traffic light in a state diagram.

The Urban traffic light goes in a continuous loop. It starts at the initial state which is at state zero and continuously goes through each state after that no matter what. Unlike the rural, with pedestrian crossing, it does not account for the pedestrian condition.

Urban with pedestrian crossing light

The Urban with pedestrian cross traffic light is just like the Urban traffic light, but it contains the pedestrian condition. From *Figure 3* if the pedestrian is sensed from either state zero or state two, it will go to a red, red condition in which the leds are red in all directions. Then it would continue on to the next state. If no pedestrian is pressed, it is treated as a normal urban traffic light, like in *Figure 3*. The Urban with pedestrian crossing light state diagram is shown below in *Figure 4*.

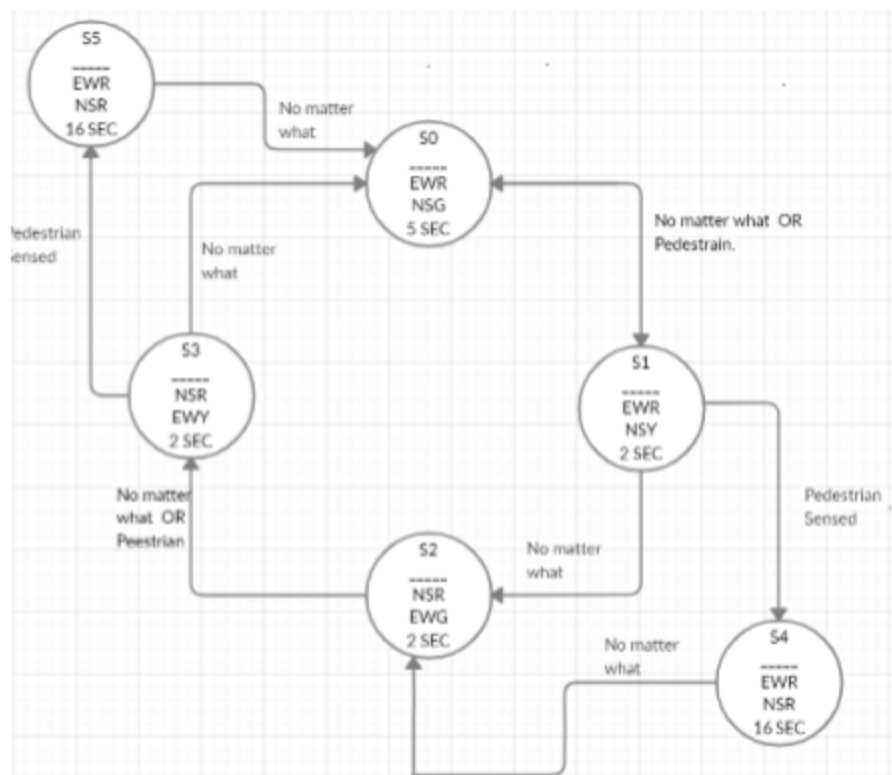


Figure 4. Shows the States of the Urban only with crosswalk traffic light in a state diagram.

Materials and ports initiation locations:

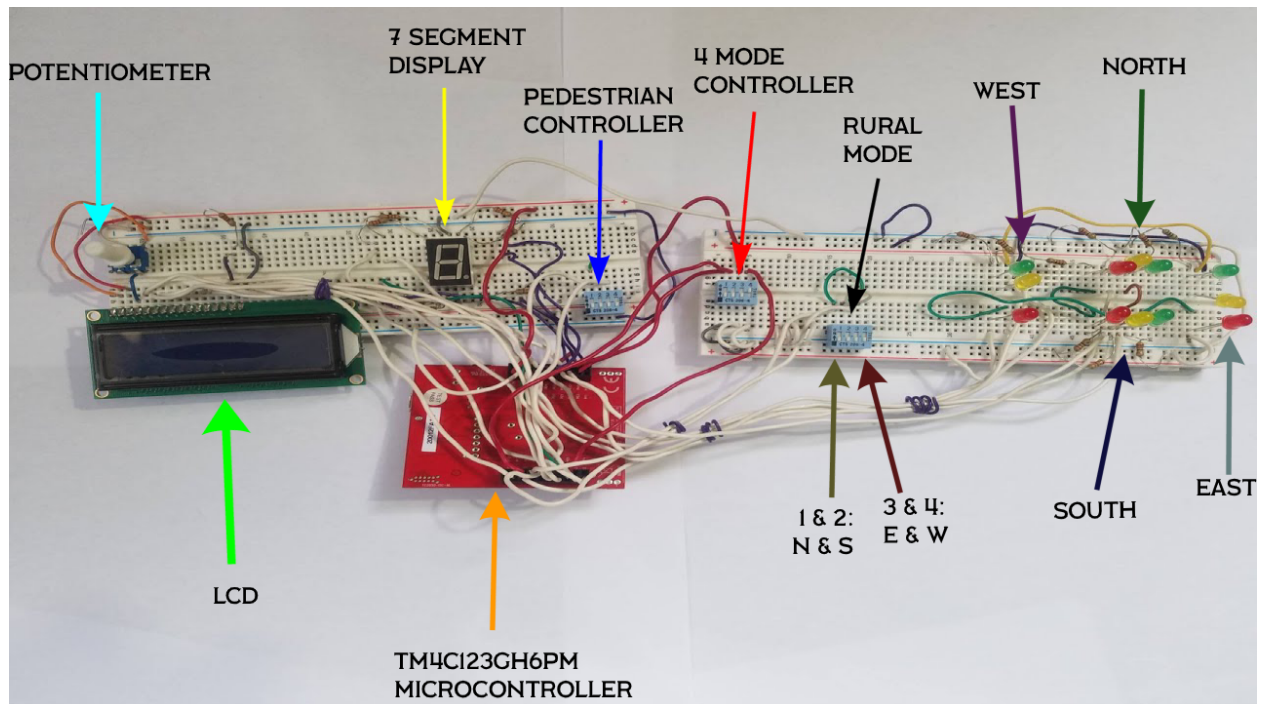


Figure 5. Diagram of the labels of each of the main components.

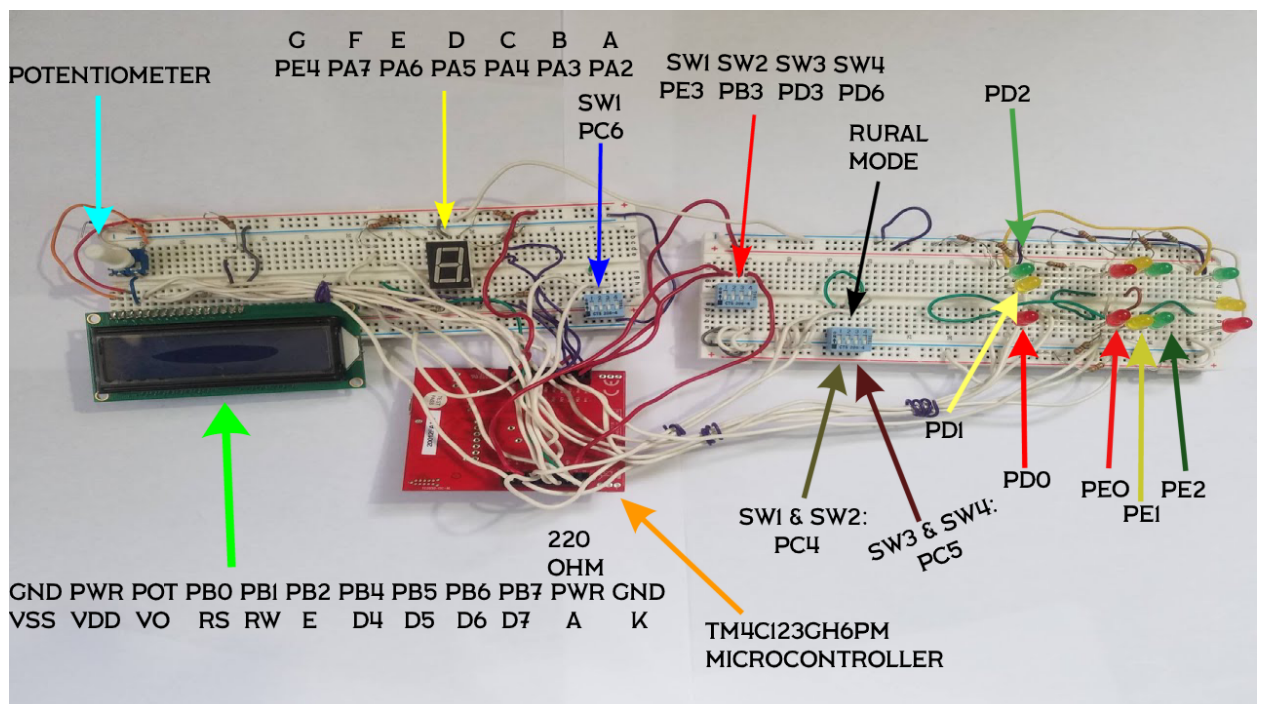


Figure 6. Diagram of the labels of where each port goes to.

Figure 5 shows the labels of each component. *Figure 6* shows where each of the component port locations are. *Figure 5* and *Figure 6* show the mechanical side of how they are wired up. All of the leds on the right side have 220 ohms resistors except for two LEDs. The two LEDs are hooked up to a 520 ohm resistor because we were out of 220 ohm resistors. For the 4 mode controller switch 1 is the Rural with Crosswalk mode, switch 2 is the Rural only mode, switch 3 is the Urban with Crosswalk mode, and switch 4 is the Urban only mode. The component labeled Rural mode is actually a Rural controller and is only used as sensors.

Code:

For the code portion we had separate methods made for each state that can be seen in all of the state diagrams above. We called those state states for the operation that were needed in the different modes. In the main loop we called the lcd commands to display the dedicated traffic light to the dedicated switch on the 4 mode controller seen in *Figure 6*. Inside the main loop, there is a while loop that has the condition to where all the four modes should go to. Multi0, Multi1, Multi2, and Multi3 variables correspond to the mode. Rural with Crosswalk mode=Multi0, Rural only mode=Multi1, Urban with Crosswalk mode=Multi2, and Multi3=Urban only mode. The variable value '2' corresponds to the first and second switch of the Rural controller. The variable value '3' corresponds to the third and fourth switch of the Rural controller. The Value1 corresponds to the program waiting for a switch to be pressed. Overall, the program goes through the main . Inside the main, it sees which mode is enabled and displays the selected traffic light, then it goes through the while loop and sees which mode is enabled and does the sequence to which that mode is supposed to do.

Problems and solutions:

The commands for the LCD were placed in the main loop, because it was causing problems in placing it in the while loop. For example, it would sometimes display a word that was supposed to go in the first row, on the second row the second iteration of the loop.

I found out that when trying to enable a particular mode the switch a set of steps were involved. First the switch had to be turned on, then the microcontroller had to be reset, and then the slide switch had to be turned off, then reset again from the microcontroller. Then the switch had to be turned on again. What I found out was that when only trying to put it on a specific mode and then placing the reset mode, it would not run all the modes. Like the red leds at the east and west side would be really dim, that one could not see it. Also both Urban modes would not go to their next state, when implementing the correct steps.

Results and Conclusion

The results were amazing. The traffic light worked as expected. All of the modes would work according to their specified state diagram. This project helped to understand different traffic lights and how state diagrams work. This project would be really useful in an actual traffic light because you can switch between any modes universally. Overall the project was a great success and met the original requirements that the project was specified to do.

References

- [1] Mylicence.sa.gov.au. 2020. *My Licence - The Driver's Handbook - Traffic Lights*. [online] Available at: <<https://mylicence.sa.gov.au/road-rules/the-drivers-handbook/traffic-lights>> [Accessed 13 May 2020].
- [2] Webpages.uidaho.edu. 2020. *Pedestrian Crossing Time, Minimum Green Interval*. [online] Available at: <https://www.webpages.uidaho.edu/niatt_labmanual/Chapters/signaltimingdesign/theoryandconcepts/PedestrianCrossingMinimumGreen.htm> [Accessed 13 May 2020].
- [3] Khan, T. and Padilla, J., 2020. *Urban And Rural Design*. pp.4-6, 18-29.
- [4] Khan, T. and Padilla, J., 2020. *Seven Segment Display and LCD* pp.3-24.

Appendices

Appendix A: Whole Project Code

```
#include "TM4C123GH6PM.h"
int ns=0;

#define LCD_PORT GPIOB
#define RS 1 /* BIT0 mask */
#define RW 2 /* BIT1 mask */
#define EN 4 /* BIT2 mask */

void delayMs(int n);
void delayUs(int n);
void LCD_nibble_write(unsigned char data, unsigned char control);
void LCD_command(unsigned char command);
void LCD_data(unsigned char data);
void LCD_init(void);

/* PORTF data register */
/* PORTF data register */
#define PORTADAT (*((volatile unsigned int*)0x400043FC))
/* PORTF data direction register */
#define PORTADIR (*((volatile unsigned int*)0x40004400))
/* PORTF digital enable register */
#define PORTADEN (*((volatile unsigned int*)0x4000451C))
/* run mode clock gating register */
#define RCGCGPIO2 (*((volatile unsigned int*)0x400FE608))
/* coprocessor access control register */

#define PORTBDAT (*((volatile unsigned int*)0x400053FC))
/* PORTF data direction register */
#define PORTBDIR (*((volatile unsigned int*)0x40005400))
/* PORTF digital enable register */
#define PORTBDEN (*((volatile unsigned int*)0x4000551C))
/* run mode clock gating register */
```

```
#define PORTEDAT (*((volatile unsigned int*)0x400243FC))
/* PORTF data direction register */
#define PORTEDIR (*((volatile unsigned int*)0x40024400))
/* PORTF digital enable register */
#define PORTEDEN (*((volatile unsigned int*)0x4002451C))
/* run mode clock gating register */

#define PORTDDAT (*((volatile unsigned int*)0x400073FC))
/* PORTF data direction register */
#define PORTDDIR (*((volatile unsigned int*)0x40007400))
/* PORTF digital enable register */
#define PORTDDEN (*((volatile unsigned int*)0x4000751C))

#define PORTCDAT (*((volatile unsigned int*)0x400063FC))
/* PORTF data direction register */
#define PORTCDIR (*((volatile unsigned int*)0x40006400))
/* PORTF digital enable register */
#define PORTCDEN (*((volatile unsigned int*)0x4000651C))

#define PORTCPUR (*((volatile unsigned int*)0x40006510))

#define SCB_CPAC (*((volatile unsigned int*)0xE000ED88))

#define PORTDPUR (*((volatile unsigned int*)0x40007510))

#define PORTEPUR (*((volatile unsigned int*)0x40024510))

#define PORTBPUR (*((volatile unsigned int*)0x40005510))

// #define PORTFPUR (*((volatile unsigned int*)0x40025510))
```

```
//void delayMs(int n);    /* function prototype for delay */
```

```
int value0=0;
int value1=0;
int value2=0;
int value3=0;
int value4=0;
```

```
int multi0=0;
int multi1=0;
int multi2=0;
int multi3=0;
int multi4=0;
```

```
int sulti0=0;
int sulti1=0;
int sulti2=0;
int sulti3=0;
int sulti4=0;
```

```
int state=0;
void s0(void)
{
    state=0;
```

```
    ns=1;
```

```
    \
```

```
    PORTDDAT=0X01;
    PORTEDAT=0X04;
    delayMs(5000);
```

```
}
```

```
void s1 ()
```

```
{
```

```
    state=1;
```

```

    PORTDDAT=0X01;
    PORTEDAT=0X02;
    delayMs(2000);

}

void s2 ()
{
    state=2;
    //ns=3;

    PORTDDAT=0X04;
    PORTEDAT=0X01;
    delayMs(5000);

}

void s3 ()
{
    state=3;

    PORTDDAT=0X02;
    PORTEDAT=0X01;
    delayMs(2000);

}

void s4 ()
{
    state=4;

    //9
    PORTADAT = 0xEC;
    //PORTCDAT = 0x11;
    //PORTE1DAT= PORTEDAT;
    PORTEDAT =0x11;

```

```

                                PORTDDAT =0x01;
delayMs(1000);

//8                                /* write PORTF to turn off all LEDs */
                                PORTADAT = 0xFF;
                                PORTEDAT = 0x11;
delayMs(1000);

//7                                /* write PORTF to turn off all LEDs */
PORTADAT = 0xC0;
                                PORTEDAT = PORTEDAT & 0xFF;
delayMs(1000);

//6                                /* write PORTF to turn on all LEDs */
PORTADAT = 0x7C;
                                PORTEDAT =0x11;
delayMs(1000);

//5    /* write PORTF to turn off all LEDs */
PORTADAT = 0x6C;
                                PORTEDAT = 0x11;
delayMs(1000);

//4                                /* write PORTF to turn on all LEDs */
PORTADAT = 0xCC;
                                PORTEDAT =0x01;
delayMs(1000);

//3    /* write PORTF to turn off all LEDs */
PORTADAT = 0xE4;
                                PORTEDAT = 0x11;
delayMs(1000);

//2                                /* write PORTF to turn on all LEDs */
PORTADAT = 0xB4;
                                PORTEDAT = 0x11;
delayMs(1000);

//1    /* write PORTF to turn off all LEDs */

```

```
PORTADAT = 0xC0;
```

```
PORTEDAT = 0x01;
```

```
delayMs(1000);
```

```
//0  /* write PORTF to turn on all LEDs */
```

```
PORTADAT = 0xF8;
```

```
PORTEDAT = 0x11;
```

```
delayMs(1000);
```

```
PORTADAT = 0x0 ;
```

```
PORTEDAT = 0x01;
```

```
delayMs(1000);
```

```
PORTADAT = 0xF8;
```

```
PORTEDAT = 0x11;
```

```
delayMs(1000);
```

```
PORTADAT = 0x0 ;
```

```
PORTEDAT = 0x01;
```

```
delayMs(1000);
```

```
PORTADAT = 0xF8;
```

```
PORTEDAT = 0x11;
```

```
delayMs(1000);
```

```
PORTADAT = 0x0 ;
```

```
PORTEDAT =0x01;
```

```
delayMs(1000);
```

```
PORTADAT = 0xF8;
```

```
PORTEDAT = 0x11;
```

```

delayMs(1000);

PORTADAT = 0x0 ;
PORTEDAT =0x01;

}

void s5 ()
{
    state=5;

    /* write PORTF to turn on all LEDs */
    //9
    PORTADAT = 0xEC;
    PORTEDAT =0x11;
    PORTDDAT =0x01;

    delayMs(1000);

    //8
    PORTADAT = 0xFF;
    PORTEDAT = 0x11;

    delayMs(1000);

    //7
    PORTADAT = 0xC0;
    PORTEDAT = PORTEDAT & 0xFF;

    delayMs(1000);

    //6
    PORTADAT = 0x7C;
    PORTEDAT =0x11;

    delayMs(1000);

    //5
    PORTADAT = 0x6C;
    PORTEDAT = 0x11;

```

```

delayMs(1000);

//4          /* write PORTF to turn on all LEDs */
PORTADAT = 0xCC;
PORTEDAT = 0x01;
delayMs(1000);

//3          /* write PORTF to turn off all LEDs */
PORTADAT = 0xE4;
PORTEDAT = 0x11;
delayMs(1000);

//2          /* write PORTF to turn on all LEDs */
PORTADAT = 0xB4;
PORTEDAT = 0x11;
delayMs(1000);

//1          /* write PORTF to turn off all LEDs */
PORTADAT = 0xC0;
PORTEDAT = 0x01;

delayMs(1000);

//0          /* write PORTF to turn on all LEDs */
PORTADAT = 0xF8;
PORTEDAT = 0x11;
delayMs(1000);

PORTADAT = 0x0 ;
PORTEDAT = 0x01;
delayMs(1000);

PORTADAT = 0xF8;
PORTEDAT = 0x11;
delayMs(1000);

PORTADAT = 0x0 ;

```



```

        value3 = PORTCDAT;
        value3=~value3;
        value3 = value3 & 0x40;

        value4 = PORTCDAT;
        value4=~value4;
        value4 = value4 & 0x80;
    }

    int count=0;
int main(void)
{

    /* enable clock to GPIOF at clock gating register */

    RCGCGPIO2 |= 0xFF;
    //GPIO_PORTF_LOCK_R |=
    /* set PORTF pin3-1 as output pins */
    PORTEDIR = 0x17      ;
        PORTDDIR = 0x07  ;
        PORTADIR = 0xFF  ;

    /* set PORTF pin3-1 as digital pins */

    PORTCPUR = 0xFF;

        PORTEPUR = 0x08;
        PORTBPUR = 0x08;
        PORTDPUR = 0x48;

    /* enable pull up for PF4 */

    PORTEDEN = 0xFF      ;
        PORTADEN = 0xFF  ;
        PORTCDEN = 0xFF;
        PORTBDEN = 0xFF;
        PORTDDEN = 0xFF;

```

```

    sulti0 = PORTEDAT;
    sulti0=~sulti0;
    sulti0 = sulti0 & 0x0;

    sulti1 = PORTEDAT;
    sulti1=~sulti1;
    sulti1 = sulti1 & 0x08;

    sulti2 = PORTBDAT;
    sulti2=~sulti2;
    sulti2 = sulti2 & 0x08;

    sulti3 = PORTDDAT;
    sulti3=~sulti3;
    sulti3 = sulti3 & 0x08;

    sulti4 = PORTDDAT;
    sulti4=~sulti4;
    sulti4 = sulti4 & 0x40;

    if(sulti1==0x08)
    {
        LCD_init();
        LCD_command(1);    /* clear display */
        LCD_command(0x80); /* LCD cursor location */

// delayMs(500);
        LCD_data('R');
        LCD_data('U');
        LCD_data('R');
        LCD_data('A');
        LCD_data('L');

        LCD_data('&');

        LCD_data('C');
        LCD_data('R');
        LCD_data('O');
        LCD_data('S');

        LCD_data('S');

```

```

        LCD_data('W');

LCD_data('A');
LCD_data('L');
LCD_data('K');

    }

    else if(sulti2==0x08)
    {
        LCD_init();
LCD_command(1);    /* clear display */
LCD_command(0x80); /* LCD cursor location */

// delayMs(500);
LCD_data('R');
LCD_data('U');
LCD_data('R');
LCD_data('A');
LCD_data('L');

        LCD_data(' ');

LCD_data('O');
LCD_data('N');
LCD_data('L');
LCD_data('Y');
    }

    else if(sulti3==0x08)
    {
        LCD_init();
LCD_command(1);    /* clear display */
LCD_command(0x80); /* LCD cursor location */

// delayMs(500);
LCD_data('U');
LCD_data('R');
LCD_data('B');
LCD_data('A');
LCD_data('N');

        LCD_data('&');

```

```

    LCD_data('C');
    LCD_data('R');
    LCD_data('O');
    LCD_data('S');

                                LCD_data('S');
                                LCD_data('W');

    LCD_data('A');
    LCD_data('L');
    LCD_data('K');

                                }

                                else if(sulti4==0x40)
                                {
                                        LCD_init();
    LCD_command(1);    /* clear display */
    LCD_command(0x80); /* LCD cursor location */

// delayMs(500);
    LCD_data('U');
    LCD_data('R');
    LCD_data('B');

                                LCD_data('A');

    LCD_data('N');

                                LCD_data(' ');

    LCD_data('O');
    LCD_data('N');
    LCD_data('L');
    LCD_data('Y');

                                }

                                s0();

while(1)
{

                                int loop=0;
                                int press=0;

                                value0 = PORTCDAT;

```

```
value0=~value0;  
value0 = value0 & 0x0;
```

```
value1 = PORTCDAT;  
value1=~value1;  
value1 = value1 & 0x10;
```

```
value2 = PORTCDAT;  
value2=~value2;  
value2 = value2 & 0x20;
```

```
value3 = PORTCDAT;  
value3=~value3;  
value3 = value3 & 0x40;
```

```
value4 = PORTCDAT;  
value4=~value4;  
value4 = value4 & 0x80;
```

```
multi0= PORTCDAT;  
multi0=~multi0;  
multi0 = multi0 & 0x0;
```

```
multi1 = PORTEDAT;  
multi1=~multi1;  
multi1 = multi1 & 0x08;
```

```
multi2 = PORTBDAT;  
multi2=~multi2;  
multi2 = multi2 & 0x08;
```

```
multi3 = PORTDDAT;  
multi3=~multi3;  
multi3 = multi3 & 0x08;
```

```
multi4 = PORTDDAT;  
multi4=~multi4;  
multi4 = multi4 & 0x40;
```

```
if(multi1==0x08)
{
```

```
    if((state==0) & (value3==0x40 ) & (ns==1) )
    {
```

```
        s1();
```

```
        s4();
```

```
        s2();
```

```
        loop=1;
```

```
    }
```

```
else if ((state==0) & ((value2 == 0x20) ) )
{
```

```
    s1();
```

```
    s2();
```

```
}
```

```
while(value1==10)
```

```
{
```

```
    ns=3;
```

```
    loop=2;
```

```
}
```

```

if((value1==0x10) && (state==2) && (value2!=0x20))
{

    s3();
    s0();
    continue;

}

else if(value3==0x40 & state==2 & loop!=1 )
{
    ns=3;
    s3();

    s5();

    s0();
    continue;
}

}

else if(multi2==0x08)
{

    if ((state==0) & ((value2 == 0x20) ) )
    {

        s1();
        s2();

    }
    while(value1==10)
    {

```

```
        //s2();
        ns=3;
        loop=2;

    }

    if((value1==0x10) && (state==2) && (value2!=0x20))
    {

        s3();

        s0();

        continue;

    }

}

else if(multi3==0x08)

{

    if((state==0) & (value3==0x40 ) & (ns==1) )
    {

        s1();

        s4();

        s2();
        loop=1;

    }

}
```

```

else
{
    s1();
    s2();
}

while(value1==10)
{
    ns=3;
    loop=2;
}

if(value3==0x40 & state==2 & loop!=1 )
{
    ns=3;
    s3();
    s5();
    s0();
    continue;
}
else
{
    s3();

    s0();
    continue;
}
}

else if(multi4==0x40)
{
    s1();
    s2();
    s3();
    s0();
}

```

```

    }

}

void LCD_init(void)
{
    SYSCCTL->RCGCGPIO |= 0x02; /* enable clock to GPIOB */
    LCD_PORT->DIR = 0xFF; /* set all PORTB pins as output */
    LCD_PORT->DEN = 0xFF; /* set all PORTB pins as digital pins */

    delayMs(20); /* initialization sequence */
    LCD_nibble_write(0x30, 0);
    delayMs(5);
    LCD_nibble_write(0x30, 0);
    delayUs(100);
    LCD_nibble_write(0x30, 0);
    delayUs(40);

    LCD_nibble_write(0x20, 0); /* use 4-bit data mode */
    delayUs(40);
    LCD_command(0x28); /* set 4-bit data, 2-line, 5x7 font */
    LCD_command(0x06); /* move cursor right */
    LCD_command(0x01); /* clear screen, move cursor to home */
    LCD_command(0x0F); /* turn on display, cursor blinking */
}

void LCD_nibble_write(unsigned char data, unsigned char control)
{
    data &= 0xF0; /* clear lower nibble for control */
    control &= 0x0F; /* clear upper nibble for data */
    LCD_PORT->DATA = data | control; /* RS = 0, R/W = 0 */
    LCD_PORT->DATA = data | control | EN; /* pulse E */
    delayUs(0);
    LCD_PORT->DATA = data;
    LCD_PORT->DATA = 0;
}

```

```

void LCD_command(unsigned char command)
{
    LCD_nibble_write(command & 0xF0, 0); /* upper nibble first */
    LCD_nibble_write(command << 4, 0); /* then lower nibble */

    if (command < 4)
        delayMs(2); /* commands 1 and 2 need up to 1.64ms */
    else
        delayUs(40); /* all others 40 us */
}

```

```

void LCD_data(unsigned char data)
{
    LCD_nibble_write(data & 0xF0, RS); /* upper nibble first */
    LCD_nibble_write(data << 4, RS); /* then lower nibble */

    delayUs(40);
}

```

/* delay n milliseconds (16 MHz CPU clock) */

```

void delayMs(int n)
{
    int i, j;
    for(i = 0 ; i < n; i++)
        for(j = 0; j < 3180; j++)
            {} /* do nothing for 1 ms */
}

```

/* delay n microseconds (16 MHz CPU clock) */

```

void delayUs(int n)
{
    int i, j;
    for(i = 0 ; i < n; i++)
        for(j = 0; j < 3; j++)
            {} /* do nothing for 1 us */
}

```

/* This function is called by the startup assembly code to perform system specific initialization tasks. */

void SystemInit(void)

{

/* Grant coprocessor access */

/* This is required since TM4C123G has a floating point coprocessor */

SCB->CPACR |= 0x00f00000;

}